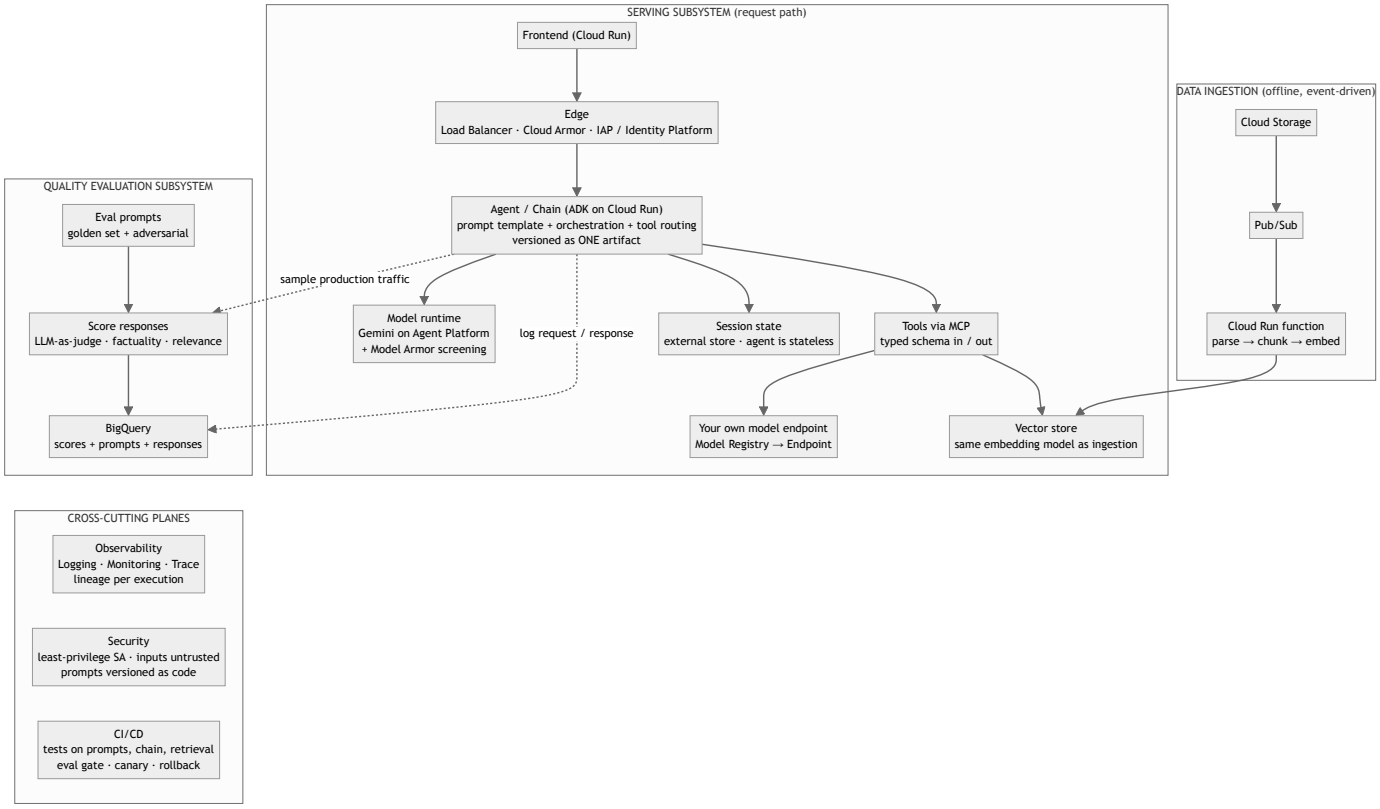
Full-Stack AI Application — Architecture Requirements

Derived from Google Cloud Architecture Center guidance · compiled 2026-09-11

1. Sources

Tag	Document (Google Cloud Architecture Center)	Last reviewed
OPS	Deploy and operate generative AI applications	2024-11-19
WAF-OE	Well-Architected Framework, AI/ML perspective: Operational excellence	2025-04-28
WAF-SEC	Well-Architected Framework, AI/ML perspective: Security	2025-11-26
RAG	RAG infrastructure for generative AI (Agent Platform + AlloyDB)	2026-02-04
COMP	Choose your agentic AI architecture components	2026-04-21
PAT	Choose a design pattern for your agentic AI system	2026-05-28
SA	Single-agent AI system using ADK and Cloud Run (reference architecture)	2025-12-09

2. The architecture Google describes



3. Requirements

MUST = Google states it as required/recommended for any production deployment. **SHOULD** = recommended, with Google noting it depends on scale or risk.

A. Components (COMP, SA)

#	Requirement	Level
A1	Frontend never calls the model directly; it talks to the agent over an API. Production frontend needs streaming support, a **stateless API**, and externalized conversation state.	**MUST**
A2	Agent built with an agent framework (ADK recommended; others acceptable) and deployed on an agent runtime: Cloud Run, Agent Runtime, or GKE.	**MUST**
A3	Agent is **stateless**; session state lives in an external store (Cloud SQL / Firestore / Memorystore / Agent Platform Sessions) so any instance can serve any request and restarts don't lose context.	**MUST**
A4	Tools exposed through **MCP** (or custom function tools when no MCP server exists), each with typed input and output schemas.	**MUST**
A5	Tool definitions kept small: primitive types, **< 5 parameters**, enums instead of free text. Unbundle monolithic tool servers into focused toolsets.	**MUST**
A6	Model served from a managed model runtime (Agent Platform / Vertex) — a dependency, not something you host.	**MUST**
A7	Long-term (cross-session) memory in an external store (Memory Bank) — only if the product needs personalization across sessions.	**SHOULD**

B. RAG subsystems (RAG, OPS)

#	Requirement	Level
B1	**Three separate subsystems**: data ingestion, serving, quality evaluation. They share the database layer; they do not share code paths.	**MUST**
B2	Ingestion is event-driven and offline: upload → Pub/Sub → function → parse → chunk → embed → vector store. Live requests never mutate the index.	**MUST**
B3	Serving uses **the same embedding model and parameters** as ingestion.	**MUST**
B4	Serving screens responses through responsible-AI / safety filters before returning to the user.	**MUST**
B5	Serving logs request/response to Cloud Logging and loads responses to BigQuery for offline analysis.	**MUST**
B6	Index maintenance (rebuild/refresh) is a scheduled operational task, versioned alongside the dataset (BigQuery / AlloyDB / Cloud Storage).	**MUST**

C. The chain is the artifact (OPS)

#	Requirement	Level
C1	The **prompted model component** (prompt template + model version) is the smallest deployable unit. Prompt template, chain definition, tool wiring, and model pin are **versioned together as one artifact** with its own revision history.	**MUST**

C2	Prompt parts classified: **prompt-as-code** (template, guardrails, instructions → Git, review, tests) vs **prompt-as-data** (few-shot examples, retrieved context, user query → validation, drift detection).	**MUST**
C3	Every execution logs inputs, outputs, **intermediate states of each chain step**, and the chain configuration used.	**MUST**
C4	Chains are evaluated **end-to-end**, not component-by-component.	**MUST**
C5	Step cap / max iterations on any agent loop (ReAct, review-critique) to prevent runaway cost.	**MUST**

D. Evaluation (OPS, WAF-OE, RAG)

#	Requirement	Level
D1	A **custom evaluation dataset** covering essential, average, and edge cases. Synthetic ground truth is acceptable when real ground truth is missing; refine with human feedback over time.	**MUST**
D2	Evaluation is **automated** (LLM-as-judge or rubric; BLEU/ROUGE are insufficient) and metrics + approach are **frozen early** so runs are comparable.	**MUST**
D3	Eval set includes **adversarial prompts** (injection, leakage, fuzzing).	**MUST**
D4	**Continuous evaluation in production**: sample outputs, score them, store scores with prompts/responses in BigQuery; collect direct user feedback.	**MUST**
D5	Evaluation runs as a **gate in CI/CD** before deploy.	**MUST**

E. Deploy (OPS, WAF-OE)

#	Requirement	Level
E1	Version control for: prompt templates, chain code, external datasets (BigQuery/AlloyDB), adapter weights (Cloud Storage), container images (Artifact Registry).	**MUST**
E2	**CI** runs unit + integration tests on prompts, chain logic, embedded models, and retrieval — not just application code.	**MUST**
E3	**CD** tests the API in a prod-like environment including load tests before release.	**MUST**
E4	**Canary / gradual release** to a subset of users; documented **rollback** to last known-good version.	**MUST**
E5	Own models (e.g. a classifier) go through a pipeline: train → evaluate → **Model Registry** → endpoint, with lineage in ML Metadata.	**MUST**
E6	Retraining/re-tuning triggered by monitoring, not by hand.	**SHOULD**

F. Observability (OPS, WAF-OE, SA)

#	Requirement	Level
F1	**End-to-end lineage**: a bad answer must be traceable to the exact prompt version, model version, index version, and tool results that produced it.	**MUST**
F2	Structured logging in the agent: which tools were called, inputs/outputs, latency per step. Distributed tracing (Cloud Trace) across frontend → agent → tools → model.	**MUST**
F3	Monitor **at the application level first**, then per component.	**MUST**
F4	Track latency, error rate, **429 rate**, token usage, request volume; alert on thresholds.	**MUST**
F5	**Skew and drift detection** on inputs vs. the eval set: text length, token counts, embedding distance, topic shifts.	**MUST**

F6	**No PII or confidential data in logs**; use Sensitive Data Protection with Logging if needed.	**MUST**

G. Security (WAF-SEC, SA)

#	Requirement	Level
G1	Treat **all inputs as untrusted** — user, retrieved documents, tool results. Filter and validate external content **before** it enters a prompt (indirect injection).	**MUST**
G2	**Layered defence**: screen prompts and responses for injection, jailbreak, harmful content (Model Armor or equivalent).	**MUST**
G3	Prompts treated as code: versioned in Git, centrally stored, reviewed, auditable.	**MUST**
G4	**Least-privilege service accounts** per service; service-to-service auth with workload identity, not shared keys.	**MUST**
G5	Front door: **external Application Load Balancer** (disable default `run.app` URL), **Cloud Armor** for rate limiting/DDoS, **IAP** (internal users) or **Identity Platform / Firebase Auth** (external users).	**MUST**
G6	**Data minimization**: prefer synthetic or de-identified data; don't collect what the product doesn't need.	**MUST**
G7	Supply chain: images built by Cloud Build, scanned in Artifact Registry, only approved images deployable (Binary Authorization / SLSA).	**MUST**
G8	**Red-team against OWASP LLM Top 10**; regular audits.	**MUST**
G9	AI-aware **incident response playbook**: rollback model/prompt version, disable endpoint, isolate data source.	**MUST**
G10	**Human-in-the-loop** checkpoint for high-stakes or safety-critical decisions.	**MUST (clinical)**

H. Reliability, cost, performance (SA, WAF-OE, COMP)

#	Requirement	Level
H1	Retries, timeouts, exception handling, and **429 handling** on model calls.	**MUST**
H2	Baseline **QPS and tokens/sec** before launch; monitor after.	**MUST**
H3	Start with the **cheapest model that passes eval**, then escalate. Control **thinking budget**; route simple tasks to smaller models.	**MUST**
H4	Concise prompts; use **context caching** for repeated high-token context.	**SHOULD**
H5	Simulate failures / load before production.	**MUST**

I. Design pattern (PAT)

#	Requirement	Level
I1	Define requirements before picking a pattern: task shape, latency tolerance, cost budget, need for human judgment.	**MUST**
I2	**Start with a single agent** and refine prompt/tools before adding agents. Multi-agent only when one agent measurably fails on tool selection or task completion.	**MUST**
I3	Every loop pattern has an explicit exit condition (max iterations, quality threshold).	**MUST**

4. What Google explicitly marks as optional / scale-dependent

VPC Service Controls perimeter · CMEK · Confidential Computing · Apigee / API Gateway · GKE over Cloud Run · Provisioned Throughput · Memory Bank / long-term memory · multi-agent patterns · fine-tuning / continuous tuning · Terraform-managed infra · multi-region.

> These are things a reviewer might *ask about*, but Google does not require them for a defensible production system. Deciding which to include is a product decision, not an architecture gap.